
Analysis of Reinforcement Learning Across Varied Environments

Vedant Narayanswami
Roll No: EP25B040

M Nharen
Roll No: EE25B089

January 8, 2026

Abstract

This report summarizes our participation in the Reward Rush Competition across a diverse set of reinforcement learning environments, including Frozen Lake, Taxi, Stochastic Frozen Lake, Lunar Lander, CartPole, and Mountain Car Continuous. We employed both on-policy and off-policy methods—ranging from classical tabular Q-learning and Q-value iteration to Deep Q-Networks (DQN), REINFORCE, and Soft Actor-Critic (SAC)—to balance sample efficiency, convergence speed, and training stability. Owing to substantial variation in state spaces, action spaces, and reward structures, we adopted task-specific solutions rather than a single generalized agent.

1 Introduction

Reinforcement learning (RL) provides a principled framework for sequential decision-making under uncertainty, enabling agents to learn control policies through interaction with environments. The effectiveness of any RL algorithm depends critically on the structural properties of the task, including the dimensionality of state and action spaces, reward sparsity, and transition stochasticity.

The Reward Rush Competition presents a diverse benchmark suite spanning discrete navigation problems, stochastic environments, and continuous control tasks. Rather than pursuing a single generalized agent, we adopt a problem-driven approach, selecting and tailoring algorithms to the underlying characteristics of each environment.

This report presents a comparative analysis of classical tabular methods, value-based deep reinforcement learning, and policy-gradient approaches across six environments. Through systematic experimentation, we demonstrate how algorithmic assumptions, exploration strategies, and function approximation choices influence convergence behavior, stability, and final performance.

2 Methodology Overview

Reinforcement Learning Formulation

All environments considered in this work are modeled as Markov Decision Processes (MDPs), formally defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ the action space, P the transition dynamics, R the reward function, and $\gamma \in [0, 1)$ the discount factor. The environments span discrete and continuous state spaces as well as discrete and continuous action spaces. This structural diversity directly dictates algorithm selection: tabular methods are suitable for small discrete domains, whereas function approximation and policy-gradient-based methods are required for continuous or high-dimensional settings.

Problem #	Environment	Algorithm(s)
1	Frozen Lake	Classical Q-learning
2	Taxi	Classical Q-learning
3	Stochastic Frozen Lake	Classical Q-learning, Q-value Iteration
4	Lunar Lander	Deep Q-Network (DQN)
5	CartPole	DQN, REINFORCE
6	Mountain Car Continuous	Soft Actor-Critic (SAC)
7	Half Cheetah	Soft Actor-Critic (SAC)
8	Walker2D	Soft Actor-Critic (SAC)

Table 1: Problem Summary: Environments and Corresponding RL Algorithms

Environment	Algorithm	γ	Learning Rate	Episodes
Frozen Lake	Q-learning	0.95	0.8	10,000
Taxi	Q-learning	0.95	0.8	10,000
Stochastic Frozen Lake	Q-learning / Q-Value Iteration	0.99	0.01	20,000
Lunar Lander	DQN	0.99	1×10^{-3}	10,000
CartPole	DQN	0.99	1×10^{-3}	10,000
Mountain Car Continuous	SAC	0.99	3×10^{-4}	1,500
Half Cheetah	SAC	0.99	3×10^{-4}	1,000
Walker2D	SAC	0.99	3×10^{-4}	4,000

Table 2: Summary of key hyperparameters used across environments

3 Problems

3.1 Frozen Lake

3.1.1 Environment Description

The classic Frozen Lake is a discrete grid-based navigation task in which an agent must navigate from a fixed starting state to a fixed goal state while avoiding holes. The state space corresponds to cells on a 4×4 grid, numbered from 0 to 15. At each time step, four discrete actions allow movement to neighboring cells.

The environment features a sparse reward structure: the agent receives a reward of 1 upon reaching the goal and 0 for all other transitions. Episodes terminate when the agent reaches the goal or falls into a hole.

The primary challenge is effective exploration, as the agent must reach the goal multiple times to learn from the sparse reward signal.

3.1.2 Algorithm

We employed classical Q-learning to train the agent, as both the state and action spaces are finite and sufficiently small to fit in a lookup table for Q-values.

Training was conducted over 10,000 episodic interactions with the environment. Actions were selected using an ϵ -greedy policy, initialized with $\epsilon = 1$ (fully random exploration) and decayed to a minimum of $\epsilon = 0.05$ with a decay rate of 0.0001. The learning rate was set to 0.8 and the discount factor $\gamma = 0.95$. Each episode was limited to a maximum of 100 steps. Q-values were initialized to zero and updated via temporal-difference learning after each step.

The resulting Q-values converged to an optimal policy that reliably avoids holes and follows the shortest safe path to the goal.

3.1.3 Results

The agent was evaluated over 100 independent test episodes. The agent achieved a 100% success rate, consistently following the optimal path and reaching the goal with the minimum number of steps required.

Details are available at: [Frozen Lake Results](#).

3.2 Taxi

3.2.1 Environment Description

The Taxi environment is a discrete grid-based navigation and planning task where the agent controls a taxi to pick up and drop off passengers at designated locations while minimizing time steps. The state is encoded as a single integer capturing taxi position, passenger location, and destination.

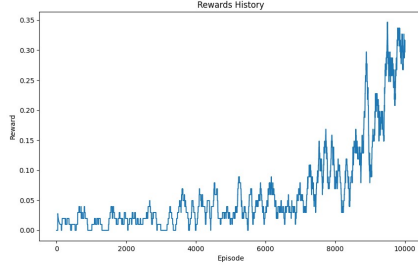


Figure 1: Training rewards per episode

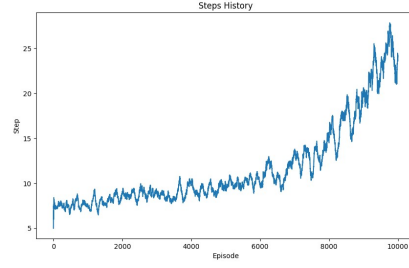


Figure 2: Steps per episode during training

At each time step, the agent selects an action to move to a neighboring cell or pick up/drop off passengers. The reward structure provides positive rewards for successful deliveries and penalties for invalid actions. Episodes terminate upon successful passenger drop-off.

Unlike Frozen Lake, the Taxi environment provides dense feedback through step penalties and action constraints, shifting the learning objective toward optimizing action sequences and ensuring action validity.

3.2.2 Algorithm

We applied classical Q-learning, as the number of states and actions remain sufficiently small for representation in a lookup table.

Hyperparameters matched the Frozen Lake setup: learning rate of 0.8, discount factor $\gamma = 0.95$, and ϵ -greedy exploration decaying from $\epsilon = 1$ to 0.05 with a decay rate of 0.0001. Each episode was limited to a maximum of 100 steps. Training over 10,000 episodes ensured reliable convergence.

The trained model adapts the taxi’s behavior based on the pick-up and drop-off locations in any given cell.

3.2.3 Results

Evaluation Protocol The trained agent was evaluated over 100 independent test episodes using the standard environment termination conditions.

Quantitative Performance

- **Success Rate:** 100% (100/100 correct passenger drop-offs).
- **Average Reward:** 7.89; the agent consistently avoided illegal moves (such as failed pickups or drop-offs) and followed efficient paths to minimize time-step penalties.
- **Efficiency:** The agent successfully reached the terminal state in every trial without exceeding the environment’s maximum step limit.

Qualitative Observations The agent demonstrated stable and efficient navigation, reliably completing passenger deliveries and minimizing penalties, with robust policy convergence across all test episodes.

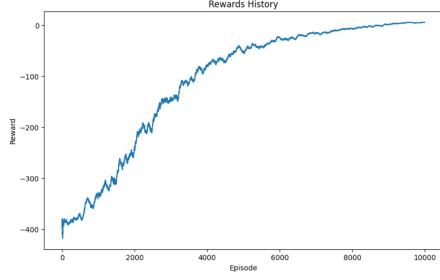


Figure 3: Training rewards over episodes

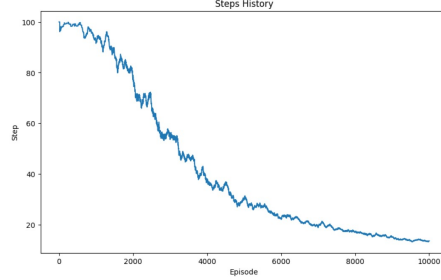


Figure 4: Steps per episode during training

Details are available at: [Taxi Results](#).

3.3 Stochastic Frozen Lake

3.3.1 Environment Description

The Stochastic Frozen Lake environment presents a discrete grid-based navigation task with decision-making under uncertainty. The agent must navigate from a starting point to a goal cell across a frozen surface with holes. Similar to deterministic Frozen Lake, the state space is represented by a single integer.

At each time step, the agent selects a direction, but stochastic transitions may cause the agent to move differently than intended, requiring a robust policy that accounts for the risk of falling into holes.

The reward structure is sparse: the agent receives a reward of 1 only upon reaching the goal. Therefore, efficient exploration and a higher discount factor are essential to overcome sparse reinforcement.

3.3.2 Algorithm

1. **Classic Q-learning:** Training was conducted over 20,000 episodes with a maximum of 100 steps per episode. We used a learning rate of 0.01, discount factor $\gamma = 0.99$, and ϵ -greedy exploration decaying from $\epsilon = 1$ to 0.05 with a decay rate of 0.00005. The lower learning rate and slower epsilon decay help handle the sparse rewards and stochastic transitions.
2. **Q-value iteration:** A model-based approach that first obtains the world model (hole locations) via Monte Carlo simulations. Transition probabilities from the environment documentation are used for 2,000 synchronous Bellman backups between successive Q-tables.

3.3.3 Results

Both algorithms were evaluated over 100 independent episodes. Classic Q-learning achieved a 0.82 success rate with an average of 39.46 steps per episode, while Q-value iteration achieved 0.76 success with 40.79 average steps. Both algorithms converged to nearly identical policies that prioritize safety, with outcome variance primarily due to environment stochasticity rather than policy quality.

Algorithm	Success Rate	Avg. Steps
Classic Q-Learning	0.82	39.46
Q-Value Iteration	0.76	40.79

Table 3: Performance comparison on Stochastic Frozen Lake

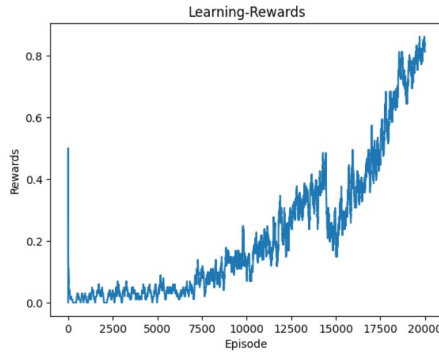


Figure 5: Q-learning reward history

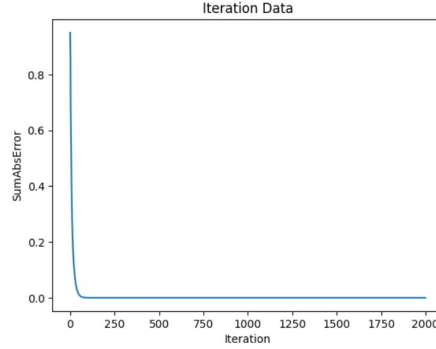


Figure 6: Q-value iteration training data

Details are available at: [Stochastic Frozen Lake Results](#).

3.4 Lunar Lander

3.4.1 Environment Description

The Lunar Lander environment is a rocket trajectory control problem where the agent must navigate a spacecraft to a landing zone between two flags at coordinates $(0,0)$ while minimizing fuel consumption. The state space is an 8-dimensional continuous vector comprising horizontal and vertical coordinates, linear velocities, orientation, angular velocity, and binary leg contact indicators.

Four discrete actions are available: no thrust, left engine, right engine, or main engine. Positive rewards are given for moving toward and landing safely on the pad, with penalties for crashes and engine use. Episodes terminate when the lander crashes, lands, or leaves the view.

This environment has a continuous state space and discrete action space, presenting the learning challenge of balancing trajectory precision with fuel efficiency.

3.4.2 Algorithm

Lunar Lander requires a Deep Q-Network (DQN) due to its continuous state space. The network maps the 8-dimensional state vector to Q-values for four discrete actions using two hidden layers of 32 ReLU units.

Training employed experience replay with a buffer of 10,000 transitions, sampling mini-batches of 256 for gradient updates. The agent trained for 10,000 episodes. We used a learning rate of 1×10^{-3} and a discount factor $\gamma = 0.99$.

3.4.3 Results

Evaluation Protocol The trained agent was evaluated over 100 independent test episodes using the standard environment termination conditions.

Quantitative Performance

- **Average Return:** 260.18; the agent consistently performed controlled landings and balanced fuel efficiency with landing precision.
- **Maximum Return:** 307.05; demonstrating the agent’s ability to achieve near-optimal trajectories under favorable initial conditions.
- **Minimum Return:** 59.66; reflecting occasional suboptimal landings due to challenging initial states or exploration noise.
- **Stability:** Standard deviation of rewards was 26.65, indicating that the agent typically landed safely with some variability.

Qualitative Observations The agent demonstrated reliable landing performance, efficiently balancing trajectory control and fuel use, with rare crashes and mostly safe landings across diverse test conditions.

Details are available at: [Lunar Lander Results](#).

3.5 CartPole

3.5.1 Environment Description

CartPole is an inverted pendulum control problem where the agent balances a pole on a cart by applying horizontal forces. The state is a 4-dimensional continuous vector (cart position, cart velocity, pole angle, pole angular velocity). Two discrete actions apply fixed forces left or right. The agent receives +1 reward per step the pole remains within a critical angle.

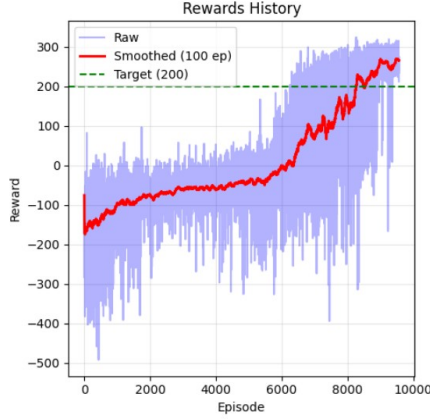


Figure 7: Training rewards over episodes

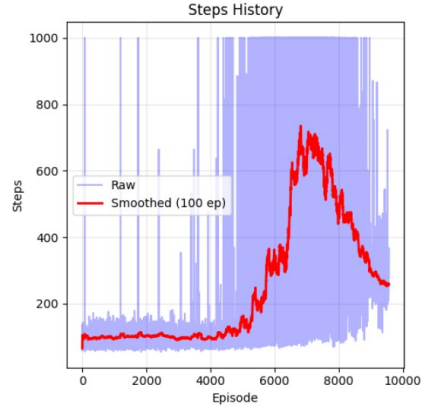


Figure 8: Steps per episode during training

Episodes terminate if the pole exceeds $\pm 12^\circ$ from vertical, the cart moves more than 2.4 units from center, or 500 steps elapse. This task demands precise control and rapid feedback to maintain balance.

3.5.2 Algorithm

We initially attempted REINFORCE but found high variance led to unstable performance. Instead, we adopted DQN with a network mapping the 4-dimensional state to Q-values for two discrete actions using two hidden layers of 128 ReLU units.

Training employed experience replay with a buffer of 10,000 transitions, sampling mini-batches of 256 for updates over 10,000 episodes. We used a learning rate of 1×10^{-3} and a discount factor $\gamma = 0.99$.

3.5.3 Results

Evaluation Protocol The trained agent was evaluated over 100 independent test episodes using the standard environment termination conditions.

Quantitative Performance

- **Average Return:** 500.00; the agent consistently maintained the pole in an upright position for the maximum possible duration.
- **Maximum Return:** 500.00; demonstrating optimal performance by reaching the environment’s episode length limit.
- **Minimum Return:** 500.00; showing no performance degradation across evaluation episodes and confirming complete policy convergence.

Qualitative Observations The agent developed a stable policy applying precise corrective actions to maintain balance, achieving perfect performance and demonstrating robust convergence.

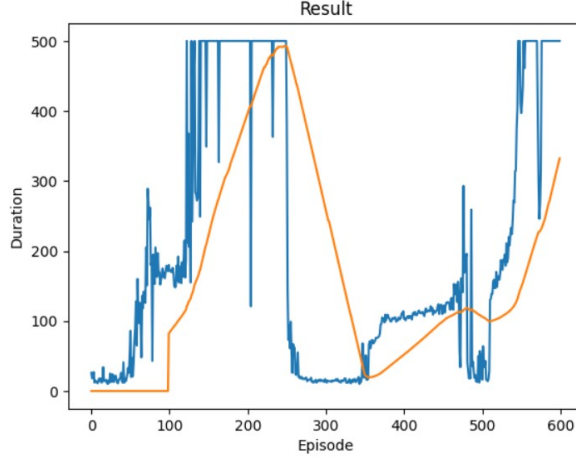


Figure 9: CartPole DQN training data showing rewards and episodes

Details are available at: [CartPole Results](#).

3.6 Mountain Car Continuous

3.6.1 Environment Description

Mountain Car Continuous is a reinforcement learning problem that extends the classic Mountain Car task to a continuous action space, increasing the complexity of the control problem. The objective is to drive an underpowered car from the bottom of a valley to the goal position at the top of the right hill.

The state space is continuous and two-dimensional, consisting of the car’s position and velocity. Unlike the discrete version, the action space is continuous and one-dimensional, where the agent selects a real-valued acceleration force within a bounded range.

Due to insufficient engine power, the car cannot directly ascend the hill and must instead learn to use momentum by applying alternating acceleration forces. The reward function penalizes energy usage while providing a positive reward upon reaching the goal state, encouraging both efficient and successful trajectories. An episode terminates when the car reaches the goal position or when the maximum episode length is exceeded.

The greatest challenge in this environment is the delayed positive reinforcement, which necessitates effective exploration.

3.6.2 Algorithm

Soft Actor-Critic (SAC) is an entropy-regularized actor-critic algorithm designed for continuous control. SAC maintains an actor network (stochastic policy) outputting action distributions and critic networks estimating soft Q-values. The policy maximizes expected cumulative reward while maximizing entropy, promoting exploration and stability.

The actor outputs Gaussian distribution parameters from the continuous state. Critic networks estimate Q-values for state-action pairs. Training uses experience replay to update both networks.

Training Hyperparameters Learning rate 3×10^{-4} ; discount factor $\gamma = 0.99$; soft-update $\tau = 0.005$; automatic entropy tuning (init $\alpha = 0.2$); replay buffer 100,000; batch size 256; start steps 1,000; episodes 1,500 (max 999 steps/episode).

3.6.3 Results

Over 100 test episodes, the agent achieved an average return of 96.07 (SD: 0.99), maximum of 97.05, and minimum of 92.90. The low variance indicates consistent, reliable goal completion through learned momentum-based strategies.

Details are available at: Mountain Car Results.

3.7 Half Cheetah

3.7.1 Environment Description

Half Cheetah is a continuous control benchmark where the agent controls a planar bipedal robot to maximize forward velocity. The state comprises joint positions and velocities; the action space is continuous, representing joint torques. Rewards encourage forward motion while penalizing excessive control effort.

3.7.2 Algorithm

We used Soft Actor-Critic (SAC) with automatic entropy tuning for this high-dimensional continuous control task. The actor and twin critic networks are two-layer MLPs with 256 hidden units per layer and ReLU activations. Actions are sampled from a squashed (tanh) Gaussian policy, and the target entropy is set to $-|\mathcal{A}|$.

Training hyperparameters (from the accompanying SAC implementation) were: learning rate 3×10^{-4} for actor, critics, and temperature optimizers; discount factor $\gamma = 0.99$; soft-update coefficient $\tau = 0.005$; replay buffer size 1,000,000; mini-batch size 256; and a warm-up of 1,000 random steps before learning. Training ran for 1,000 episodes with periodic evaluations and checkpoints.

3.7.3 Results

The agent achieved consistent forward locomotion with stable gait patterns across all 100 test episodes. While absolute performance did not reach reported benchmark maxima, the learned policy demonstrated effective coordination and robust performance.

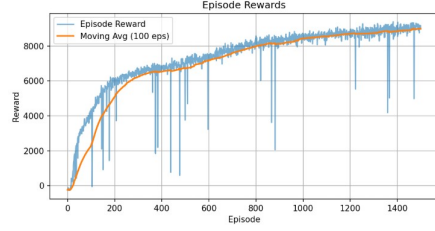


Figure 10: Reward per episode

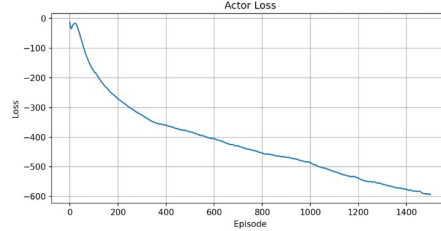


Figure 11: Actor network loss

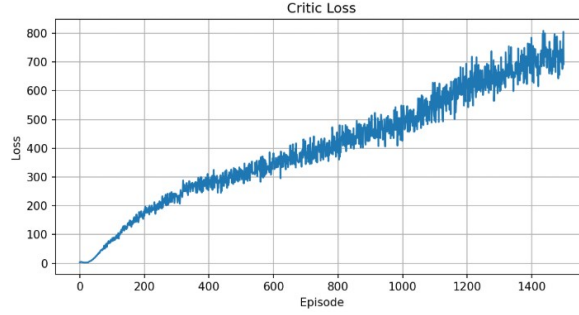


Figure 12: Critic network loss

Details are available at: [Half Cheetah Results](#).

3.8 Walker2D

3.8.1 Environment Description

Walker2D is a continuous control benchmark featuring a planar bipedal robot consisting of a torso, thighs, legs, and feet. The objective is to develop a stable walking pattern that maximizes forward velocity while maintaining balance. The state space includes the positions and velocities of the torso and joints; the action space is continuous, representing the torques applied to the six joints. Rewards are granted based on forward distance progressed and are penalized for high energy consumption or if the walker falls (indicated by the torso height dropping below a specific threshold).

3.8.2 Algorithm

We utilized Soft Actor-Critic (SAC), an off-policy actor-critic algorithm based on the maximum entropy reinforcement learning framework. SAC was selected for this environment because its entropy-augmented objective improves sample efficiency and provides greater robustness to hyperparameter settings. By maximizing both expected reward and entropy, the agent maintains a diverse set of behaviors, which is critical for discovering stable walking patterns in the unstable Walker2D morphology.

Training Hyperparameters We used SAC with automatic entropy tuning and a squashed Gaussian policy. Networks are deeper for Walker2D: the actor and twin critics are 3-layer MLPs with 400 hidden units per layer and ReLU activations. Optimization used a learning rate of 3×10^{-4} for actor, critics, and temperature; discount factor $\gamma = 0.99$; soft-update coefficient $\tau = 0.005$; replay buffer size 1,000,000; mini-batch size 256; warm-up of 10,000 random steps; and one gradient update per environment step. The target entropy was set to $-|\mathcal{A}|$. Training ran for 4,000 episodes with periodic evaluations and checkpoints.

3.8.3 Results

The SAC agent successfully learned a stable bipedal walking pattern, achieving robust performance over 100 evaluation episodes. The maximum return achieved was 4432.63, the minimum return was 4210.33, and the average return was 4290.40. The standard deviation among episodes was approximately 37.65, indicating consistent performance across test runs. Unlike the Half Cheetah, the Walker2D required more precise torque control to prevent falling; however, the entropy regularization in SAC prevented premature convergence to sub-optimal, "shuffling" motions. The final policy demonstrated a fluid stride with consistent torso stabilization.

Details are available at: [Walker2D Results](#).

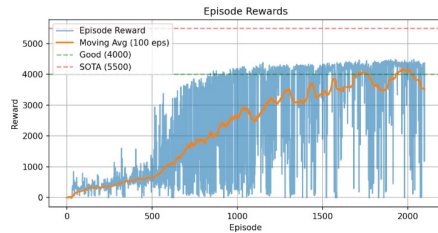


Figure 13: Walker2D episode rewards over training

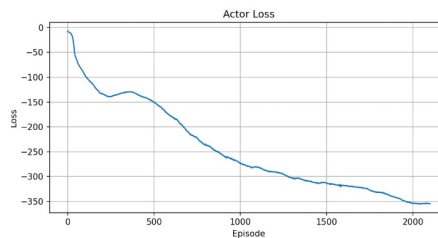


Figure 14: Walker2D actor network loss

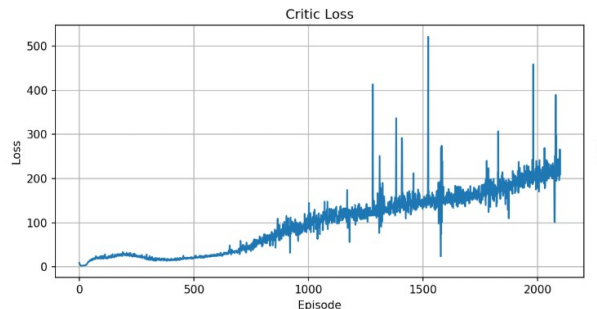


Figure 15: Walker2D critic network loss

4 Training Stability and Failure Modes

Across environments, training stability varied significantly with algorithm choice. Tabular methods exhibited reliable convergence in small discrete domains but scaled poorly to larger state spaces. Value-based deep methods such as DQN demonstrated sensitivity to exploration schedules and replay buffer composition, particularly in early training. Policy-gradient-based methods benefited from entropy regularization, which reduced premature convergence and improved robustness in continuous control tasks.

5 Conclusion

This work demonstrates that no single RL algorithm is universally optimal. Discrete, low-dimensional problems are efficiently solved using tabular methods, whereas continuous or high-dimensional domains require function approximation and policy-gradient techniques. Careful alignment of algorithm selection with environment characteristics proved critical to achieving stable, high-performing agents across all tasks. Future work could explore meta-learning approaches to automate algorithm selection, or develop hybrid methods combining the strengths of multiple approaches.